



Diplomado en Data Science Aplicada con Python para la Toma de Decisiones

Clase 36: RAG multimodal — PDFs sucios, imágenes, tablas y Chroma

Carlos Enrique Mosquera Trujillo

9 de Junio, 2026

Dónde quedamos en la clase 35



Construimos el asistente Arca que cita el **Código de Ética y Cumplimiento** y rechaza los 3 ataques. PDF de texto plano + NumPy + cosine. Funcionó.

Hoy: en planta los datos no vienen así de limpios. PDFs escaneados, fotos de pizarra, tablas en imagen. Y queremos algo que persista.

La pregunta del día

Tu RAG de clase 35 indexó un PDF de texto plano sobre Ética. Pero en planta, los datos vienen sucios: manuales escaneados, fotos del compresor, tablas de códigos de error. ¿Cómo los convertís a vectores, dónde los guardás, y armás un asistente de mantenimiento Arca que entiende todo eso?

- ▶ **Codificar** — PDF (limpio o escaneado), imagen, tabla → texto.
- ▶ **Almacenar** — vector DB (Chroma) que persiste y filtra por metadata.
- ▶ **Recuperar** — RAG multimodal sobre 3 tipos de contenido a la vez.

Nos acompaña en las 6 paradas de hoy. Cierre con respuesta.

El arco — 6 paradas hacia el RAG multimodal



- 0. **El dolor** — 3 casos donde el RAG de clase 35 se cae.
- 1. **PDFs** — texto extraíble vs escaneado. PyPDF o Scout OCR.
- 2. **Imágenes** — captioning con Llama 4 Scout.
- 3. **Tablas** — breve, 3 enfoques.
- 4. **Chroma** — el primer vector DB de verdad.
- 5. **Asistente Arca** — el chatbot de mantenimiento.

Bloque o

El dolor:
por qué el RAG de clase 35 se rompe

3 casos donde el RAG de clase 35 falla



PDF escaneado

PyPDF devuelve vacío



imagen del compresor

no es texto



tabla de códigos error

pierde estructura

El RAG de clase 35 daba por sentado: **(1)** el PDF tiene texto incrustado extraíble, **(2)** todo el contenido es texto, **(3)** no hay estructura tabular. En planta, las 3 asunciones se rompen el primer día.

Demo: misma pregunta, dos corpus, dos resultados

```
texto_limpio    = "".join(p.extract_text() for p in
    ↪ PdfReader("manual.pdf").pages)
texto_escaneado = "".join(p.extract_text() for p in
    ↪ PdfReader("manual_ESCANEADO.pdf").pages)

print(f"limpio:      {len(texto_limpio)} chars")      # → 3624
print(f"escaneado:   {len(texto_escaneado)} chars")   # → 0 (!!)
```

**Output real ejecutado: PyPDF extrae 3624 caracteres del PDF nativo, cero del escaneado. Mis-
mo libro, dos archivos, dos universos.**

El RAG de clase 35 sobre el escaneado responde *“no aparece”* a todo — porque *no encuentra ningún chunk*. Necesitamos otra ruta para extraer.

🏠 ¿Qué me llevo a Arca? — Bloque 0

Verdad incómoda:

- ▶ Un **PDF nativo** (Word → Exportar PDF) tiene texto incrustado. PyPDF lo extrae en milisegundos.
- ▶ Un **PDF escaneado** (foto/scanner → PDF) es una imagen disfrazada. PyPDF devuelve vacío.
- ▶ En planta Arca, **los manuales útiles suelen ser escaneados de los años 90**. Si tu chatbot sólo procesa PDFs nativos, sirve para Ética pero NO para mantenimiento.

El RAG de clase 35 funciona si el PDF es limpio. En planta no lo es. → Bloque 1: una sola función pdf_a_texto() que decide.

Bloque 1

Codificar PDFs:
nativos y escaneados, mismo pipeline

Anatomía mínima de un PDF

PDF NATIVO

generado por Word, LaTeX,
exportador de software

objeto "Text" incrustado

PDF ESCANEADO

foto/scanner → PDF
o página renderizada

objeto "Image", sin texto

Heurística rápida: si al abrir el PDF podés seleccionar texto con el mouse → es nativo. Si te selecciona un rectángulo → es escaneado. PyPDF respeta esa misma frontera.

PDF nativo — PyPDF en 3 líneas

```
from pypdf import PdfReader

texto = "\n".join(p.extract_text() for p in
    ↳ PdfReader("manual.pdf").pages)
print(texto[:200])
# Manual de Mantenimiento – Planta Embotelladora
# Arca Continental Ecuador • Versión 2026.06
# 1. Compresor Atlas Copco GA-160 VSD
```

Mismo patrón que clase 35. Sale texto limpio listo para chunkear y embeber.

PDF escaneado — PyPDF devuelve nada

```
texto = "".join(p.extract_text() for p in  
    ↪ PdfReader("manual_ESCANEADO.pdf").pages)  
print(len(texto), "caracteres extraídos")  
# 0 caracteres extraídos
```

Cero caracteres. PyPDF no inventa texto, lee lo que está en el PDF. En el escaneado el contenido vive como *imagen* dentro de cada página.

Necesitamos un paso previo: convertir cada página a una imagen (pymupdf) y pedirle a un **modelo de visión (VLM)** que “lea” la imagen y devuelva texto. Eso es OCR moderno.

Paso intermedio — página → imagen con pymupdf

```
import fitz, base64
doc = fitz.open("manual_ESCANEADO.pdf")
pix = doc[0].get_pixmap(dpi=140)           # renderiza pagina 1
png_bytes = pix.tobytes("png")
b64 = base64.b64encode(png_bytes).decode()
print(f"PNG: {len(png_bytes)} bytes, base64: {len(b64)}  
↪ chars")
```

- ▶ pymupdf (módulo fitz): renderiza cada página como mapa de pixels.
- ▶ dpi=140 es buen balance calidad/tamaño para OCR.
- ▶ **Base64** convierte los bytes binarios a string para mandar al API.

Scout como VLM OCR — anatomía del request

```
client.chat.completions.create(  
    model="meta-llama/llama-4-scout-17b-16e-instruct",    # VLM,  
    ↪ no chat normal  
    messages=[{"role": "user", "content": [  
        {"type": "text", "text": "OCR de esta imagen. Solo  
        ↪ el texto."},  
        {"type": "image_url", "image_url": {  
            "url": f"data:image/png;base64,{b64}"  
        }},  
    ]}],  
    max_tokens=900, temperature=0,  
)
```

Mismo cliente OpenAI-compatible de clase 34-35. Cambia el model y el formato del content: una lista con *texto + image_url* (que acepta data: URI con base64).

Output real de Scout OCR

Página 1 del manual ESCANEADO (PyPDF = 0 chars)

(la pagina se ve como una imagen en el PDF)

Respuesta REAL de Scout (714 chars)

*"Manual de Mantenimiento — Planta Embotelladora
Arca Continental Ecuador — Versión 2026.06*

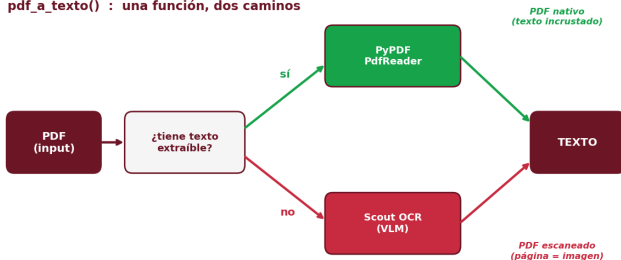
1. Compresor Atlas Copco GA-160 VSD

El compresor GA-160 VSD es la fuente principal de aire comprimido de la Línea 2. Opera a presión nominal de 8.5 bar con frecuencia variable..."

☒ **Recupera presión nominal (8.5 bar), modelo (GA-160 VSD), sección, número de línea. Listo para chunkear.**

Pipeline universal — pdf_a_texto()

pdf_a_texto() : una función, dos caminos



Una función, dos caminos. **Heurística:** ¿PyPDF devolvió <80 chars? → caer a OCR con Scout. Si no → usar el texto que vino.

Cuándo usar cada uno

Tipo de PDF	Mejor herramienta	Costo aprox
nativo (Word, LaTeX)	PyPDF	free, <1 s por PDF
escaneado limpio	Scout OCR	~0.5 s por página, free tier Groq
escaneado borroso	Scout OCR con prompt cuidadoso	idem + revisión humana
formularios complejos	Cloud Doc AI / Textract	\$0.01-0.05 por página
PDF protegido / cifrado	romper protección antes	manual

💡 **Default: PyPDF primero, Scout de fallback. Cubrés el 95 % de los casos en Arca con esa lógica.**

🏠 ¿Qué me llevo a Arca? — Bloque 1

El lunes ya puedes:

- ▶ **Ingestar un manual escaneado** de los años 90 sin tirarlo a la basura.
- ▶ Una sola función `pdf_a_texto()` que decide entre PyPDF y Scout OCR.
- ▶ Costo: **free tier Groq alcanza** para procesar miles de páginas mensuales.

PDF con texto → PyPDF. PDF escaneado → VLM. Mismo pipeline. → Bloque 2: imágenes sueltas (no en PDF).

Bloque 2

Imágenes: captioning con VLM
(porque hasta ahora todo era texto)

Dos paradigmas para indexar imágenes

Approach	Cómo	Mejor para
(a) Embedding visual (CLIP)	imagen → vector directo (mismo espacio que texto)	“buscame fotos parecidas a esta foto”
(b) Captioning + texto	imagen → descripción → embed texto	“mi RAG ya es de texto y quiero meter imágenes”

Hoy vamos por (b). Llama 4 Scout describe la imagen en texto, la metés en el *mismo* corpus de texto, y la búsqueda semántica de clase 35 sigue funcionando sin tocar nada del retrieval.

Por qué *captioning* gana cuando el RAG ya es de texto

- ▶ **Reusás** el mismo embedder (mpnet) y la misma Chroma. Cero infraestructura nueva.
- ▶ **Mezclás** en el mismo top-K: una foto del compresor y un párrafo del manual quedan en el mismo espacio.
- ▶ El VLM **razona sobre la imagen** en español: lee códigos de error, identifica modelos de equipo, describe colores y estados.
- ▶ Costo **predecible**: una sola llamada por imagen al ingerir, cero llamadas extra al *query*.

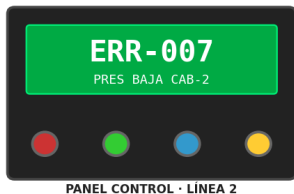
Trade-off honesto: si el contenido visual es *muy* importante (colores, formas), CLIP gana. Para mantenimiento donde el texto del cartel es lo decisivo, captioning gana.

Scout como *captioner* — una función

```
def caption_imagen(path, prompt="Describe esta imagen en una
↪ frase. Si ves un código de error, cítalos."):
    with open(path, "rb") as f:
        b64 = base64.b64encode(f.read()).decode()
    r = client.chat.completions.create(
        model="meta-llama/llama-4-scout-17b-16e-instruct",
        messages=[{"role": "user", "content": [
            {"type": "text", "text": prompt},
            {"type": "image_url", "image_url": {"url":
↪ f"data:image/png;base64,{b64}"}}],
        ]}],
        max_tokens=150, temperature=0,
    )
    return r.choices[0].message.content.strip()
```

Idéntica forma a la de OCR. *Mismo modelo, distinto prompt.*

Demo — foto del panel de control Línea 2



prompt

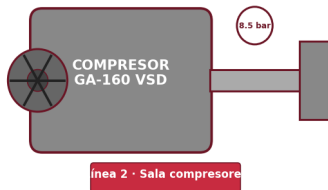
“Describe esta imagen en una frase.
Si ves código de error, cítalo.”

respuesta REAL de Scout

“La imagen muestra un panel de control con una pantalla verde que indica **ERR-007** y **PRES BAJA CAB-2**, acompañado de cuatro botones de colores: rojo, verde, azul y amarillo, bajo el titular *PANEL CONTROL · LÍNEA 2*.”

✓ Capturó el código de error, la descripción, la línea, los colores de los botones. Esa frase → embedder → vector → Chroma.

Demo — foto del compresor (otro caso)



prompt

"Describe esta imagen en una frase.
Qué modelo/equipo es?"

respuesta REAL de Scout

"La imagen muestra un compresor **GA-160 VSD**."

Corto y al punto. Si querés más detalle, pedís más en el prompt.

Truco: cuanto más específico el prompt, más útil el caption. "Lista cualquier código, marca o nivel visible" > "Describí esto".

Pipeline completo — imagen → vector

Imagen → texto → vector. Mismo espacio que el resto del corpus.



ej.: "Compresor GA-160 VSD operando a 8.5 bar en la sala de compresores de la Línea 2."

Importante: guardamos en Chroma el caption como un documento más, con `metadata={"modalidad": "imagen", "path": ...}`. En el query, el resultado nos dice de dónde vino.

🏠 ¿Qué me llevo a Arca? — Bloque 2

El lunes ya puedes:

- ▶ **Indexar fotos de equipos** de las pizarras de planta y dejarlas en el mismo Chroma que los manuales.
- ▶ Usar Scout para *leer códigos de error visibles en pantalla* aunque vengan de una foto del operador con el celular.
- ▶ Pensar el prompt del captioner como una *plantilla* dependiente del dominio (mantenimiento $\neq$ marketing).

Una imagen vale mil palabras; con un VLM, vale un vector. → Bloque 3: y las tablas, sin morirnos.

Bloque 3

Tablas:
3 caminos, brevedad

3 enfoques para tablas (de menor a mayor complejidad)

Enfoque	Cómo	Cuándo
(1) tabla entera → markdown	<code>pandas.read_csv().to_markdown()</code> → 1 vector	tabla chica (<50 filas)
(2) una fila por documento	cada fila → texto → 1 vector	tabla grande, queries por fila
(3) tabla como imagen → VLM	captura → Scout describe → 1 vector	tabla en PDF escaneado

Default Arca: (2) una fila por documento. La gente pregunta por casos puntuales (“¿qué pasa con ERR-007?”), no por la tabla entera.

Ejemplo — `codigos_error.csv` fila por fila

```
import pandas as pd
df = pd.read_csv("codigos_error.csv")

docs, metas, ids = [], [], []
for _, row in df.iterrows():
    docs.append(f"{row['codigo']} ({row['equipo']}): "
               f"{row['causa']}. Accion: {row['accion']}")
    metas.append({"modalidad": "tabla", "codigo":
        ↪ row["codigo"], "equipo": row["equipo"]})
    ids.append(row["codigo"])

print(docs[6])
# 'ERR-007 (Llenadora): Perdida sostenida de presion en cabezal
↪ 2.
# Accion: Parar linea, inspeccionar sello del cabezal 2,
↪ reemplazar O-ring.'
```

Cada fila → una frase compacta. La metadata permite filtrar después:
`where={"equipo": "Llenadora"}`.

Por fila vs por tabla entera — cuándo cada uno

Una fila por doc — default

Pro: query precisa, “ERR-007” va a la fila exacta. Metadata por equipo. Escala a miles de filas.

Contra: pierde la vista global (“cuántos errores hay del compresor”).

Tabla entera — exception

Pro: una sola consulta devuelve la estructura completa.

Contra: si la tabla pasa de 50 filas, el chunk se vuelve enorme y pierde precisión semántica.

💡 Para Arca con manuales de planta: `una_fila_por_doc` y arriba un *document-resumen* con la tabla en markdown como bonus.

🏠 ¿Qué me llevo a Arca? — Bloque 3

- ▶ Tablas **NO son ciencia rara**. Default: una fila por documento.
- ▶ Si la tabla está en un PDF escaneado, podés capturarla como imagen y pasársela a Scout (mismo pipeline que en B2).
- ▶ Si la tabla es muy grande, dividís por equipo o por sección.

Para tablas: brevedad. Tres caminos, uno por defecto. → Bloque 4: el primer vector DB real — Chroma.

Bloque 4

Vector DB: Chroma
(qué te da que NumPy no)

Refresco rápido — embeddings de clase 35



- ▶ Misma idea de clase 35. paraphrase-multilingual-mpnet-base-v2.
- ▶ 768 dimensiones. Distancia coseno $\leftrightarrow$ distancia semántica.
- ▶ Hoy reusamos exactamente este embedder, sólo cambia *dónde guardamos los vectores*.

NumPy + cosine ya está — ¿por qué cambiar?

```
docs_emb = embedder.encode(chunks, normalize_embeddings=True)
→ # (N, 768)
def top_k(q, k=3):
    q_emb = embedder.encode([q], normalize_embeddings=True)[0]
    return np.argsort(-(docs_emb @ q_emb))[:k]
```

Esto SIGUE FUNCIONANDO para ≤ 1.000 docs. No es un error.

Lo que no te da: persistencia (al cerrar el notebook se pierde el índice), filtros por metadata (modalidad, equipo, fecha), ni escalado a 100k docs sin custom code.

Qué te da un vector *DB* de verdad

Capability	NumPy + cosine	Chroma
persistencia	manual (<code>np.save/pickle</code>)	SQLite + ANN index en disco
filtros metadata	~ con pandas + máscara	<code>where={"modalidad": "imagen"}</code>
incremental ingest	re-construir el array	<code>.add()</code> y listo
escalado	~1k docs OK, 100k cae	hasta millones (HNSW index)
comparte entre runs	archivo nuevo cada vez	abrís la misma colección

Chroma es el siguiente paso natural de NumPy. Sin servidor, `pip install chromadb`, vive en el mismo proceso.

Anatomía de Chroma — `.add()` y `.query()`



Dos métodos cubren el 95 % del uso. Lo demás es metadata (`where={...}`), filtros de texto (`where_document`) y operaciones de housekeeping.

.add() — el indexador en 6 líneas

```
import chromadb
client = chromadb.PersistentClient(path="./mtto_arca_db")
col = client.get_or_create_collection("mtto",
    ↪ metadata={"hnsw:space": "cosine"})

embs = embedder.encode(chunks,
    ↪ normalize_embeddings=True).tolist()
col.add(documents=chunks, embeddings=embs,
    metadata=[{"modalidad": "texto"} for _ in chunks],
    ids=[f"doc_{i}" for i in range(len(chunks))])

print(col.count(), "documentos indexados")
```

Pasamos embeddings= explícito para usar **nuestro mpnet**, no el embedder default de Chroma.
Coherente con clase 35.

.query() — recuperar con filtro

```
q = "como soluciono presion baja en el cabezal 2"
q_emb = embedder.encode([q],
    ↪ normalize_embeddings=True).tolist()

res = col.query(
    query_embeddings=q_emb, n_results=3,
    where={"modalidad": "tabla"},           # solo
    ↪ entradas de la tabla
)

for doc, meta, dist in zip(res["documents"][0],
    ↪ res["metadatas"][0], res["distances"][0]):
    print(f"[{meta['codigo']}] dist={dist:.3f} {doc[:60]}")
# [ERR-007] dist=0.32 ERR-007 (Llenadora): Perdida sostenida
    ↪ de presion...
```

El filtro where acota a la modalidad "tabla". La query semántica encuentra ERR-007 aunque la pregunta diga "cabezal 2" (no "CAB-2").

Persistencia + reabrir desde disco

```
## Día 1: ingestamos
client = chromadb.PersistentClient(path="./mtto_arca_db")
col = client.get_or_create_collection("mtto")
col.add(...)

# Día 2 (otro notebook, otra máquina):
client = chromadb.PersistentClient(path="./mtto_arca_db")
col = client.get_collection("mtto")      # ya esta indexada
print(col.count())                      # → todos los
↪ documentos siguen ahí
```

☑ **Indexás una vez, consultás para siempre.** Compatible con git si las DB son pequeñas; con S3 / volumen montado si son grandes.

Demo en vivo — query con metadata filter

Query

```
"equipo industrial vibrando", where={"fuente": "manual"}
```

Top-2 (output real ejecutado)

1. "Compresor 3 con vibracion alta a 80 Hz" **dist=0.464**
2. "Llenadora pierde presion cabezal 2" **dist=0.659**

(la política de regalos fue filtrada por where)

Sin el filtro, el RAG mezcla manuales con ética. Con el filtro, el asistente *de mantenimiento* contesta sólo con manuales.

🏠 ¿Qué me llevo a Arca? — Bloque 4

El lunes ya puedes:

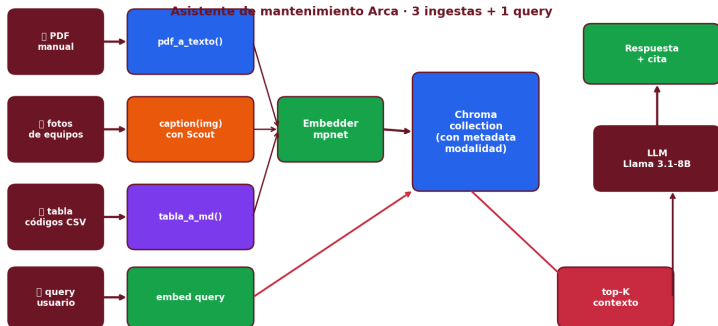
- ▶ **Migrar el chatbot de clase 35 a Chroma** en 30 minutos sin tocar el retrieval ni el embedder.
- ▶ Hacer la DB **persistente**: indexás una vez, querías todos los días.
- ▶ Filtrar por **modalidad / equipo / fecha / área** con `where={...}`.

Lo que NumPy te dejaba probar, Chroma te lo deja persistir. → Bloque 5: el asistente Arca de mantenimiento.

Bloque 5

El asistente de mantenimiento Arca:
3 ingestas + 1 query

Arquitectura completa



Cada pieza ya la viste por separado. Ahora se compone.

Ingesta 1 — texto del manual (PDF + OCR)

```
texto = pdf_a_texto("manual_mtto_arca.pdf") # PyPDF
chunks = sliding_window(texto, tam=300, overlap=60)

embs = embedder.encode(chunks,
    ↪ normalize_embeddings=True).tolist()
col.add(
    documents=chunks, embeddings=embs,
    metadatas=[{"modalidad": "texto", "fuente": "manual"} for _ in
    ↪ chunks],
    ids=[f"txt_{i}" for i in range(len(chunks))],
)
```

Si el manual viniera escaneado, la única línea que cambia es la primera: `pdf_a_texto()` detecta y cae a Scout OCR.

Ingesta 2 — fotos de equipos (Scout caption)

```
fotos = ["panel_control.png", "compresor.png",  
        ↪ "etiquetadora.png"]  
captions = [caption_imagen(p) for p in fotos]    # Scout en cada  
        ↪ foto  
  
embs = embedder.encode(captions,  
        ↪ normalize_embeddings=True).tolist()  
col.add(  
    documents=captions, embeddings=embs,  
    metadatas=[{"modalidad":"imagen","path":p} for p in fotos],  
    ids=[f"img_{p}" for p in fotos],  
)
```

3 fotos, 3 llamadas a Scout, 3 captions, 3 embeddings, 3 documents en la misma collection.

Ingesta 3 — tabla de códigos (CSV → una fila por doc)

```
df = pd.read_csv("codigos_error.csv")
filas = [
    f"{r.codigo} ({r.equipo}): {r.causa}. Accion: {r.accion}"
    for _, r in df.iterrows()
]
embs = embedder.encode(filas,
    → normalize_embeddings=True).tolist()
col.add(
    documents=filas, embeddings=embs,

    → metadatas=[{"modalidad": "tabla", "codigo": r.codigo, "equipo": r.equipo}
                  for _, r in df.iterrows()],
    ids=df["codigo"].tolist(),
)
```

Después de las 3 ingestas, la collection tiene un universo unificado: texto + imagen + tabla, todos vectores 768d.

La metadata es el truco — modalidad por documento

Modalidad	Documents (en Chroma)	Metadata extra
texto	chunks del manual	fuelle=manual, seccion=...
imagen	captions de Scout	path=fotos/compresor.png
tabla	una fila por código	codigo=ERR-007, equipo=Llenadora

En el query podés decir “solo imágenes”, “solo la tabla”, “texto del compresor”. La metadata te da vistas de la misma base.

Query #1 — pregunta de texto

Pregunta del técnico

“ERR-007 que hago?”

Top-3 (Chroma) — mezcla las 3 modalidades

[tabla] ERR-007 (Llenadora): Perdida sostenida de presion en cabezal 2.

[texto] Caso típico: cabezal 2 pierde presión sostenidamente (ERR-007)...

[imagen] Panel con pantalla verde “ERR-007 PRES BAJA CAB-2”...

Respuesta del LLM (con cita)

Parar la línea. Inspeccionar el sello del cabezal 2 de la llenadora. Reemplazar el O-ring (pieza KHS-OR-024 en stock). *Fuente: tabla ERR-007 + sección 2 del manual.*

Query #2 — foto del panel + pregunta

```
## El operador subió una foto del panel
caption = caption_imagen("foto_operador.png")    # Scout
↳ describe la foto
pregunta = "Que hago?"
query = f"{caption} | {pregunta}"

res = col.query(query_embeddings=embed(query), n_results=3)
respuesta = llm(SYSTEM, build_context(res) + f"\n\nPREGUNTA:
↳ {pregunta}")
```

✓ Truco simple: convertir la foto a caption *antes* de buscar. La query queda en texto + texto, tu RAG no cambia.

Output real — caso completo

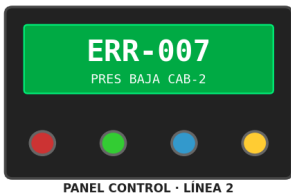


Foto que mandó el operador desde su celular

Scout caption

"Panel con ERR-007 PRES BAJA CAB-2..."

Top-K Chroma

*Tabla ERR-007 + Sección 2 manual
+ caption de su propia foto*

Respuesta

*"Detectaste ERR-007 (presión baja cabezal 2). Parar línea,
inspeccionar sello, reemplazar O-ring KHS-OR-024."*

El asistente reconoce el código en la foto y devuelve el procedimiento del manual.

El notebook trae todo esto en 6 piezas

1. **pdf_a_texto(path)** — PyPDF o Scout OCR según contenido.
2. **caption_imagen(path)** — Scout describe la imagen.
3. **tabla_a_docs(csv)** — pandas + una fila por documento.
4. **ingest(col, manual, fotos, tabla)** — las 3 ingestas en una llamada.
5. **rag(pregunta, foto=None)** — query texto o multimodal.
6. **Gradio** — interface con upload de imagen.

Gradio en 8 líneas

```
import gradio as gr

def chatbot(pregunta, foto):
    return rag(pregunta, foto=foto)

gr.Interface(
    fn=chatbot,
    inputs=[gr.Textbox(label="Pregunta"),
            gr.Image(type="filepath", label="Foto"
                    ↳ (opcional))],
    outputs=gr.Textbox(label="Respuesta"),
    title="Asistente Mantenimiento Arca",
).launch(share=True)
```

URL pública de Colab, lo mostrarás al supervisor el martes. Sin despliegue.

¿Qué me llevo a Arca? — Bloque 5

- ▶ Un asistente que **lee texto, imágenes y tablas** del mismo manual.
- ▶ Se entiende con el técnico de planta *en su lenguaje* — texto o foto.
- ▶ Vive en Chroma persistente. Lo entrenás una vez, lo usás todos los días.

Un solo asistente lee texto, imagen y tabla del mismo manual. → Bloque 6: lo armás en el no-tebook ahora.

Bloque 6

Build en vivo + PBL

Cómo abrir el notebook

github.com/cmosquerat/arca-diplomado/tree/main/clase-36

Abrilo en Colab, pegá tu GROQ_API_KEY cuando getpass lo pida.

Las 6 piezas del Bloque 5 ya están en el notebook:

- ✓ Sección 1 — `pdf_a_texto()` y `caption_imagen()`.
- ✓ Sección 2 — `tabla_a_docs()`.
- ✓ Sección 3 — `ingest()` + Chroma persistente.
- ✓ Sección 4 — `rag(pregunta, foto=None)` con metadata filter.
- ✓ Sección 5 — Gradio con upload de imagen.
- ✓ Sección 6 — PBL: los 3 ejercicios siguientes.

3 ejercicios PBL — para hoy

E1 — migrar el chatbot Ética de clase 35 a Chroma

Sustituí NumPy por Chroma persistente. Recargá los chunks de Ética del notebook de clase 35. *Criterio:* las 3 queries de la sección 6.5 de clase 35 siguen funcionando.

E2 — agregá una foto tuya al corpus

Tomá una foto de cualquier equipo / placa / pantalla y captionéalo con Scout. Indexalo en la misma Chroma. *Criterio:* una query semántica relevante a tu foto la recupera en top-1.

E3 — tu dominio

Cambiá el corpus por uno tuyo: PDF + 3 imágenes + 1 CSV. Cualquier área de Arca. *Criterio:* 5 queries, una de cada tipo (texto, imagen, tabla, híbrida, fuera-de-scope).

¿Qué me llevo a Arca? — Bloque 6

- ▶ Un **notebook plantilla** multimodal listo para tu dominio.
- ▶ Una **Chroma persistente** compartida entre runs.
- ▶ 3 **ejercicios PBL** acotados que terminás en una tarde.

Lo armás esta tarde y lo mostrás el lunes. → Cierre.

Cierre

Lo que te llevas

La respuesta a la pregunta del día

Tu RAG de clase 35 indexó un PDF de texto plano sobre Ética. Pero en planta, los datos vienen sucios: manuales escaneados, fotos del compresor, tablas de códigos de error. ¿Cómo los convertís a vectores, dónde los guardás, y armás un asistente de mantenimiento Arca que entiende todo eso?

Convertir: PyPDF o Scout OCR según contenido (texto, escaneado, imagen).

Guardar: Chroma persistente con metadata por modalidad.

Armar: 3 ingestas + 1 query + LLM con cita.

Producción real = ingesta sucia + vector DB + RAG multimodal.

Checklist de producción multimodal

- ✓ 1. pdf_a_texto() con fallback automático (PyPDF → Scout OCR).
- ✓ 2. Captioning de imágenes con *prompt específico al dominio*.
- ✓ 3. Tablas: una fila por documento + metadata por equipo/código.
- ✓ 4. Chroma persistente, con metadata modalidad.
- ✓ 5. System prompt blindado heredado de clase 35.
- ✓ 6. Citas de la modalidad en la respuesta (“de tabla ERR-007 + manual”).
- ✓ 7. Re-ingest incremental cuando hay manual nuevo, sin re-construir todo.

Imprimí esta lista, pegala al lado del monitor.

Gracias

Preguntas?

Clase 36 — RAG multimodal · Asistente de mantenimiento Arca

github.com/cmosquerat/arca-diplomado/tree/main/clase-36